Informe parcial Fuga de la Prisión

Curso: Programación Orientada a Objetos · **Ciclo:** 2026-2 · **Fecha de revisión:** 15 de octubre de 2026

Integrantes: Jose Fernando Chavez Lozano; Mickele Carlos Torres Velazco; Richard Aaron Soriano Cordova; Piero Cemei Romero Meza.

I. Resumen Ejecutivo

El proyecto parcial presenta un nivel jugable de «Fuga de la Prisión»: el prisionero esquiva vigilancia, recoge una llave y llega a la salida. La arquitectura separa personajes, reglas, estados, puntaje y vista. El código demuestra asociación, agregación, composición, encapsulamiento, herencia y polimorfismo, además de State y Observer. Se entregan UML, interfaz funcional, pruebas e historial Git disponible; los distintos niveles pertenecen al proyecto final [1, pp. 1–3; 2, p. 1].

II. Introducción

a. Contexto y objetivo

El proyecto integra los contenidos de Programación Orientada a Objetos en un videojuego 2D. El tema asignado dice: «El personaje debe escapar de una prisión esquivando guardias, cámaras y sistemas de seguridad» [1, p. 2]. Para el parcial, la rúbrica valora el diagrama de clases, seis fundamentos de POO, el avance visible del juego y del código, el uso de Git en equipo y la explicación oral [2, p. 1]. El objetivo de este avance es demostrar esas decisiones en un primer pabellón jugable, sin afirmar que el producto final ya esté completo.

b. Tecnologías utilizadas

El programa está escrito en Python 3.12 y usa Pygame 2.6.1 para ventana, teclado y dibujo. Se utilizó Mermaid.js como formato editable del UML, `unittest` para comprobar reglas y Git para registrar versiones. El repositorio contiene el código, los diagramas, las capturas y los resultados de verificación. No se requiere una API para ejecutar este prototipo local.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El diagrama principal está en `docs/diagramas/modelo_parcial.mmd` y su imagen en `docs/diagramas/modelo_parcial.png`. Muestra **nueve clases reales**, por encima del umbral de ocho clases bien fundamentadas del nivel excelente [2, p. 1]. El modelo ampliado de once clases está en `modelo_completo.mmd`. Cada caja separa nombre, atributos y métodos, como se enseña en la sesión 1 [3, pp. 59–62]. Las multiplicidades indican cuántos objetos participan. El triángulo vacío representa herencia; el rombo blanco, agregación; el rombo negro, composición; la flecha continua, asociación navegable; y la flecha discontinua, dependencia [3, p. 64; 4, pp. 20–27; 5, pp. 7–19]. Las palabras de las relaciones se explican aquí, fuera del diagrama, para mantenerlo legible.

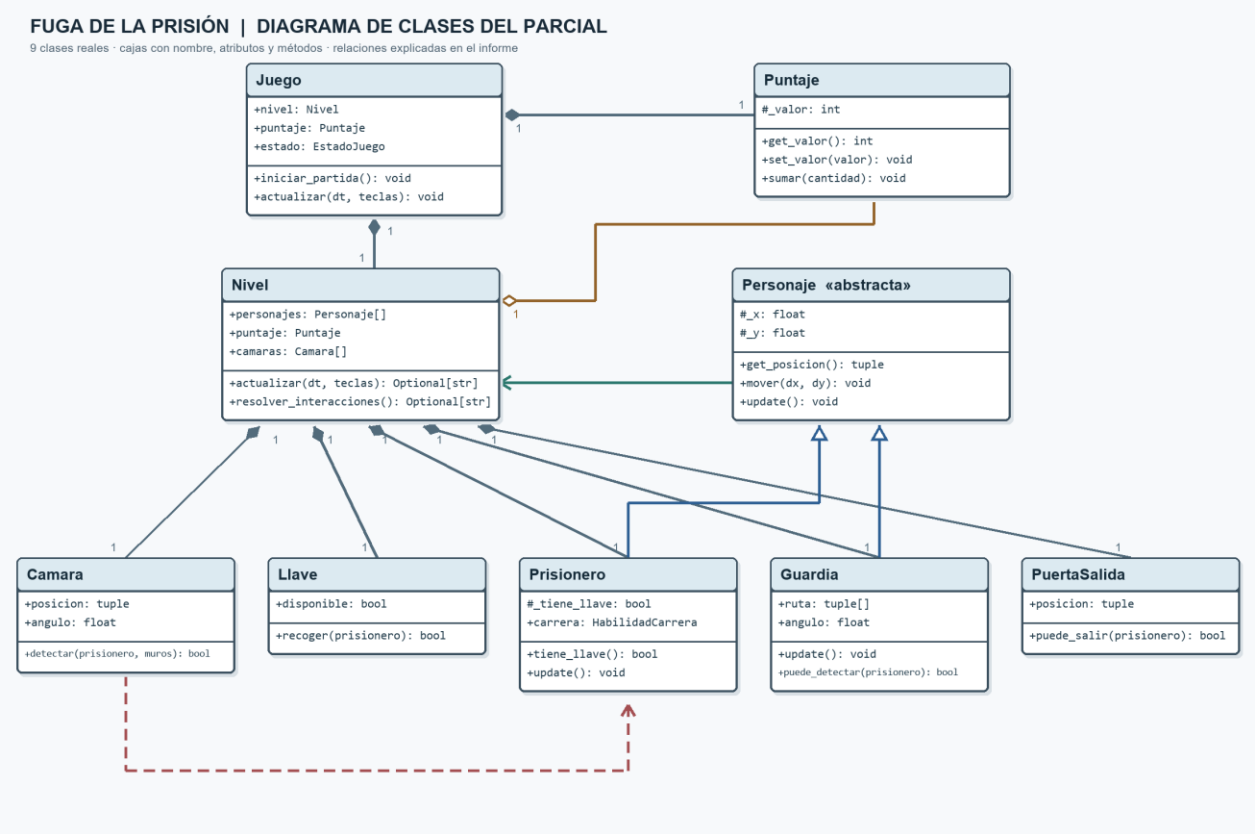


Figura 1. Diagrama UML del parcial. Fuente editable: modelo_parcial.mmd.

`Personaje` es la clase abstracta de `Prisionero` y `Guardia`. `Juego` crea un `Nivel` y un `Puntaje`. `Nivel` recibe el puntaje ya creado, por lo que su vínculo con él es agregación; en cambio, crea su prisionero, guardia, cámara, llave y salida. `Prisionero` crea `HabilidadCarrera`. Cada personaje conoce su nivel, lo que constituye asociación navegable. `Camara`, `Llave` y `PuertaSalida` reciben un prisionero en métodos concretos sin conservarlo: esas son dependencias.

b. Estructura de componentes

`Juego` coordina el estado activo, el nivel y el puntaje. `Nivel` mantiene el mapa, actualiza los personajes y resuelve detección, llave y salida. `Personaje` define posición y movimiento comunes; `Prisionero` interpreta las teclas y `Guardia` patrulla. `Camara` comprueba visibilidad; `Llave` y `PuertaSalida` implementan los dos objetivos del pabellón. `Puntaje` valida el valor y notifica cambios al panel. `Vista` dibuja el mapa, los personajes stickman, los campos de visión y la interfaz, sin decidir las reglas de victoria. Estas responsabilidades se comprueban en `src/`.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. En `Personaje.__init__`, `self.nivel = nivel` conserva una referencia al nivel. El personaje consulta ese objeto al moverse y actualizarse; no lo crea. El enlace UML es `Personaje --> Nivel`.

Agregación. `Juego.iniciar_partida()` construye `Nivel(self.puntaje)`. `Nivel.__init__` guarda el puntaje que recibió, y ese puntaje permanece en `Juego` cuando se crea otro nivel al reiniciar. Por ello, el rombo blanco está junto a `Nivel`, el todo que usa esa parte compartida. La sesión 4 distingue agregación por la independencia de la parte [5, pp. 7–10].

Composición. `Nivel.__init__` construye `Prisionero`, `Guardia`, `Camara`, `Llave` y `PuertaSalida` para su pabellón. `Prisionero.__init__` construye su propia `HabilidadCarrera`. El rombo negro está junto a `Nivel` o `Prisionero`, respectivamente. El reinicio reemplaza el nivel y crea nuevos objetos para la partida nueva. En el modelo de este juego son partes propias del todo [5, pp. 15–19].

b. Herencia y polimorfismo

`Prisionero(Personaje)` y `Guardia(Personaje)` heredan posición y movimiento. Ambos sobrescriben `update()`, exigido como método abstracto por `Personaje` [6, pp. 6 y 26–27; 8, pp. 17–18]. El bucle de `Nivel.actualizar()` no comprueba el tipo de personaje:

```
for personaje in self.personajes:
    personaje.update()
```

En el prisionero, la llamada usa teclas y habilidad de carrera; en el guardia, avanza por una ruta y orienta su visión. Esa misma operación con resultados distintos es polimorfismo por subtipo [7, pp. 27–28]. La prueba `test_actualizacion_polimorfica` comprueba que se llama `update()` en ambos objetos.

c. Encapsulamiento y abstracción

`Puntaje` guarda `_valor` como dato interno. `get_valor()` lo consulta; `set_valor(valor)` valida que sea un entero no negativo y notifica al observador; `sumar(cantidad)` acepta solo enteros positivos y usa el setter. Así, una actualización válida mantiene sincronizado el panel. `Personaje` mantiene `_x`, `_y` y `_velocidad` como datos internos y expone `get_posicion()` y `mover()`, que valida colisiones. El prefijo `_` es una convención de uso interno en Python; la protección efectiva está en los métodos de validación [9, pp. 7–12].

`Personaje` es abstracta mediante `ABC` y `@abstractmethod`: representa el contrato común sin permitir un personaje genérico instanciable. `EstadoJuego` y `ObservadorPuntaje` también son bases abstractas. La sesión 7 muestra ese mecanismo para Python [8, pp. 17–18].

Patrones de diseño implementados

State. `Juego` delega entrada y actualización a su `EstadoJuego`. `Jugando`, `Pausa`, `Derrota` y `Victoria` aplican comportamientos distintos; la pausa detiene la simulación. **Observer.** `Puntaje` conserva sus observadores y notifica a `PanelPuntaje` cuando cambia. Se implementan estos dos patrones de los tres propuestos en el proyecto final; **no** se afirma que exista Factory Method [1, pp. 2–3].

VI. Implementación de Resultados Parciales

El prototipo tiene un pabellón jugable con movimiento por WASD o flechas, colisiones, un guardia patrullando, una cámara, campos de visión bloqueados por muros, llave, salida, puntaje, carrera temporal, pausa, derrota, victoria y reinicio. El mapa obliga a entrar al cuarto de la llave, regresar por su acceso, usar cobertura para esperar al guardia, cruzar el pasillo central y rodear obstáculos antes de llegar a la salida. El atajo superior está bloqueado. La interfaz muestra objetivo, controles, tiempo, puntaje y estado de la habilidad. La llave suma 100 puntos; completar la salida suma 500. Se puede demostrar una ruta completa sin teletransportar al personaje.



Figura 2. Interfaz inicial generada por Pygame en modo de verificación.

El 15 de octubre de 2026 se ejecutó `tools/verificar.py` con Python 3.12.14 y Pygame 2.6.1: **22 pruebas, cero fallos y cero errores**. La ruta automatizada obtuvo 600 puntos en 1937 actualizaciones. También se generaron cuatro capturas con Pygame en modo SDL sin ventana y se comprobaron eventos de pausa, reanudación, pérdida de foco, reinicio y cierre. Estos resultados demuestran ejecución automatizada; aún no documentan un ensayo humano con teclado. La rúbrica menciona «25 %» para el máximo de este criterio, pero no define cómo calcular ese porcentaje [2, p. 1]; por ello se presentan funcionalidades verificables en vez de un porcentaje inventado.

Git. El historial local contiene cinco commits entre el 8 y el 15 de octubre de 2026; todos tienen el mismo autor configurado. Esto acredita uso de Git, pero **no acredita participación de todos los integrantes ni evidencia del equipo al 100 %**. Antes de la entrega, cada integrante debe registrar sus aportes reales con su identidad. El estado comprobable está en `docs/evidencias/git_estado.md`; no se fabricaron commits ni autorías.

Para el proyecto final todavía faltan distintos niveles. La sección de API del proyecto final se titula «sugerencia», aunque el cuerpo dice «debe integrar»; su obligatoriedad requiere aclaración docente [1, p. 3]. Ni los niveles posteriores ni la API se presentan como implementados en este parcial.

VII. Conclusiones

El primer pabellón permite explicar con código y ejecución las seis relaciones y fundamentos exigidos para la calificación máxima de POO. La llamada polimórfica a `update()` y la validación de `Puntaje` son ejemplos concretos que el equipo puede mostrar durante la exposición. State y Observer separan el flujo de la partida y la actualización del panel.

La mayor limitación documental es Git: el historial disponible solo identifica a un autor, por lo que la evidencia de trabajo en equipo sigue incompleta. También falta realizar y registrar una prueba manual en el equipo de exposición. El siguiente trabajo verificable consiste en incorporar las contribuciones reales del equipo y ensayar la demostración; la ampliación a varios niveles pertenece al proyecto final.

VIII. Anexos

- **Diagrama de clases:** `docs/diagramas/modelo_parcial.mmd` y `modelo_parcial.png`; modelo ampliado `modelo_completo.mmd`.
- **Código de asociación, composición y polimorfismo:** `src/personajes.py` y `src/nivel.py`.
- **Código de agregación y State:** `src/juego.py` y `src/estados.py`.
- **Código de encapsulamiento y Observer:** `src/puntuacion.py`.
- **Interfaz y funciones:** `src/vista.py` y `main.py`.
- **Pruebas y capturas:** `tests/test_juego.py` y `docs/evidencias/`.
- **Evidencia Git y límites:** `docs/evidencias/git_estado.md`.

Fuentes del curso

[1] *Proyecto Final POO*, ciclo 2026-2, pp. 1–3. Consigna entregada por el curso.

[2] *Rúbrica y Estructura - Proyecto Parcial*, ciclo 2026-2, pp. 1–3. Consigna entregada por el curso.

[3] *Sesión 1A*, «Clase, objeto, abstracción y modelado UML», pp. 59–64.

[4] *Sesión 3A*, «Asociación entre objetos», pp. 6–8 y 20–27.

[5] *Sesión 4A*, «Agregación y Composición», pp. 7–10 y 15–25.

[6] *Sesión 5A*, «Herencia y Sobrescritura de Métodos», pp. 6 y 26–27.

[7] *Sesión 6A*, «Polimorfismo», pp. 27–28.

[8] *Sesión 7A*, «Comparación de la POO en Lenguajes de Programación», pp. 17–18.

[9] *Sesión 2A*, «Encapsulamiento», pp. 7–12.